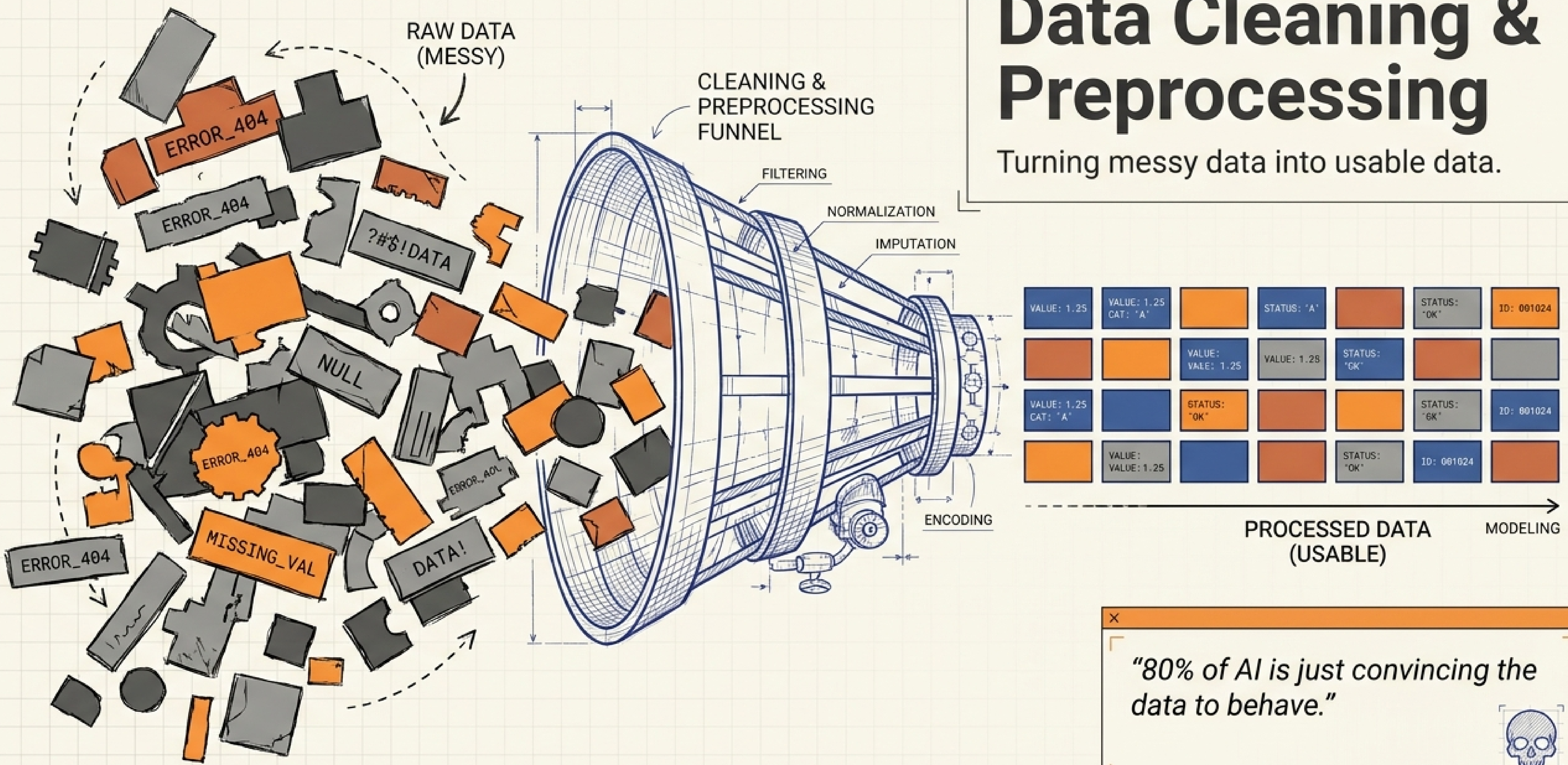
Data Cleaning & Preprocessing

Turning messy data into usable data.



"80% of AI is just convincing the data to behave."

A model can be mathematically brilliant and still fail because the data was garbage.

ID	Age	City	Salary
001	28	San Fran	55000
002	35	London	62000
003	42	**New York**	71000
004	**-5**	Boston	48000
005	25	**N.Y.**	**“42”**
006	NaN	Chicago	**NaN**

Integer stored as a string

Missing value gap

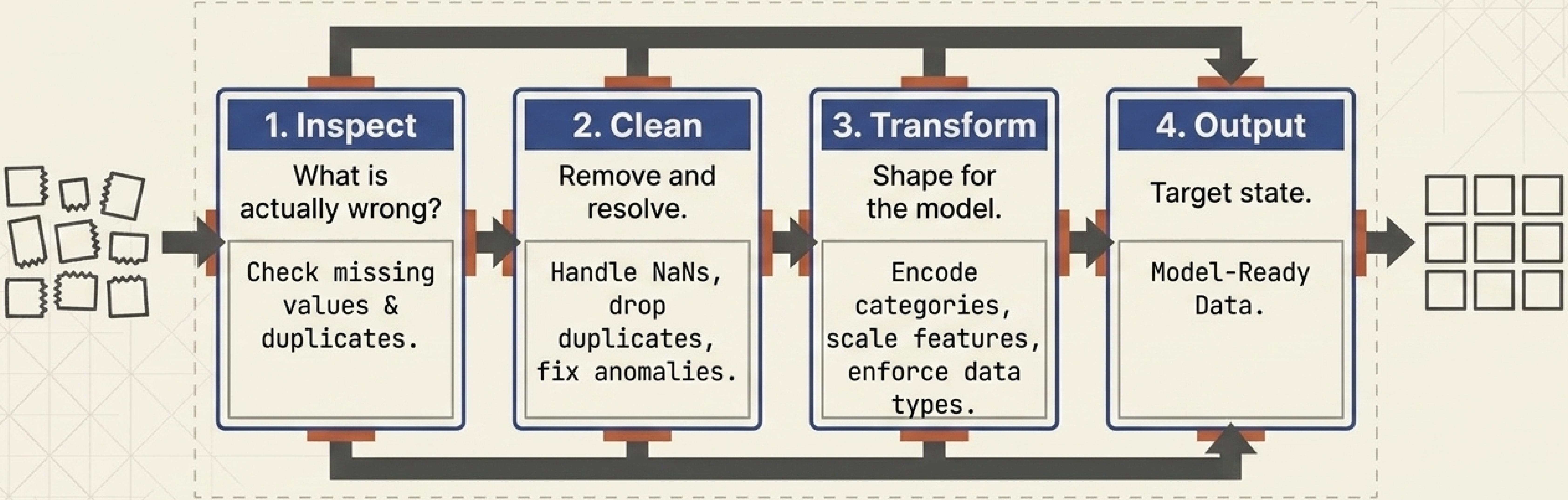
Impossible domain error

Inconsistent categories

Can we directly **train** a model on this? **Usually not.**

The Data Preparation Pipeline

Objective: Convert raw data into meaningful model-ready features.



Inspect Before Cleaning

Never start changing the dataset blindly. First question: What is actually wrong with this dataset?

Missing Values

Are there gaps represented by NaN (Not a Number)?

[ID: 004] [Age: 25] [Salary: NaN]

Duplicates

Are records repeating and skewing influence?

[ID: 002] [Age: 35] [City: London] [Salary: 62000]

[ID: 002] [Age: 35] [City: London] [Salary: 62000]

Data Types

Are mathematical numbers hiding as text strings?

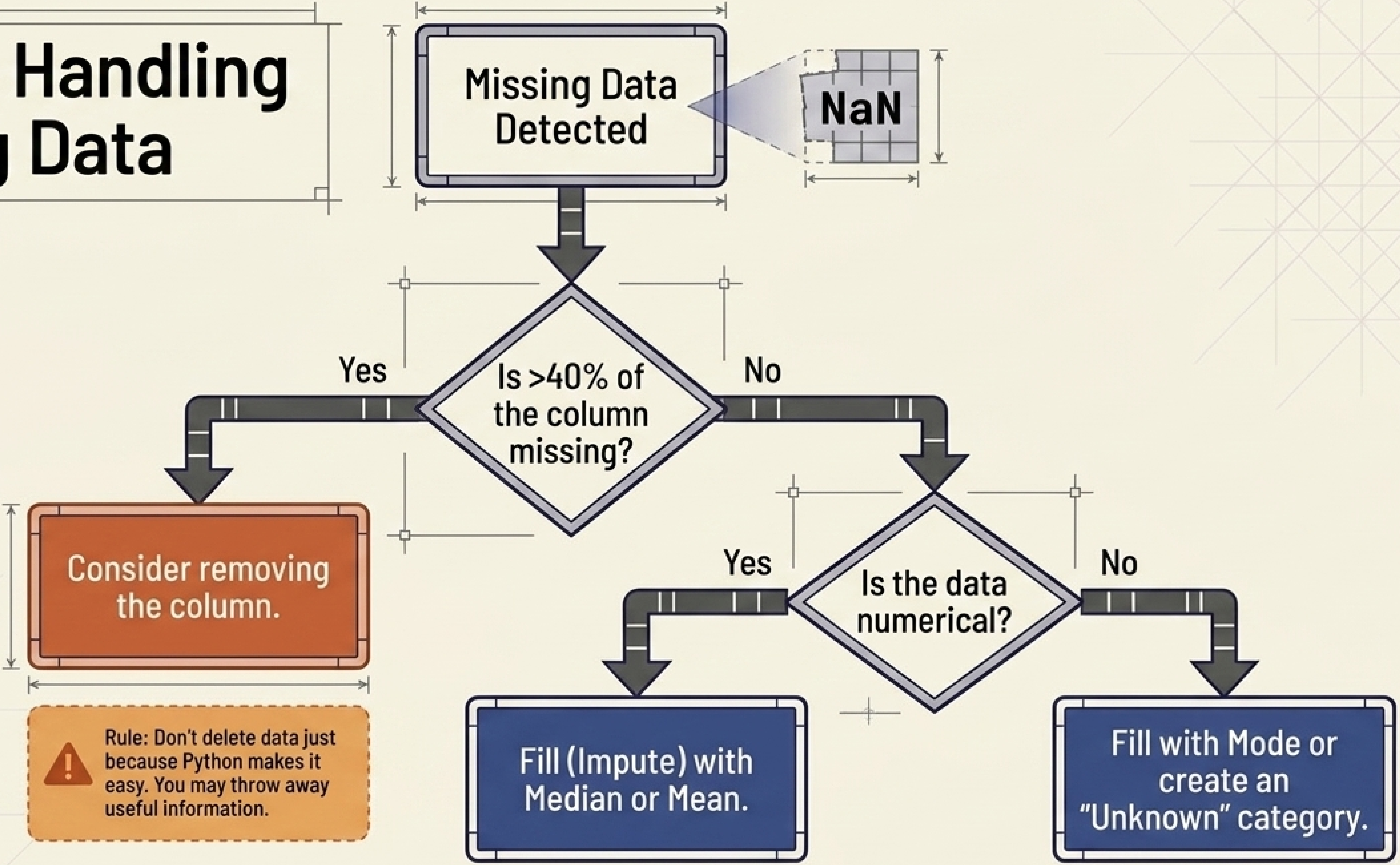
String Type

"100" → 100

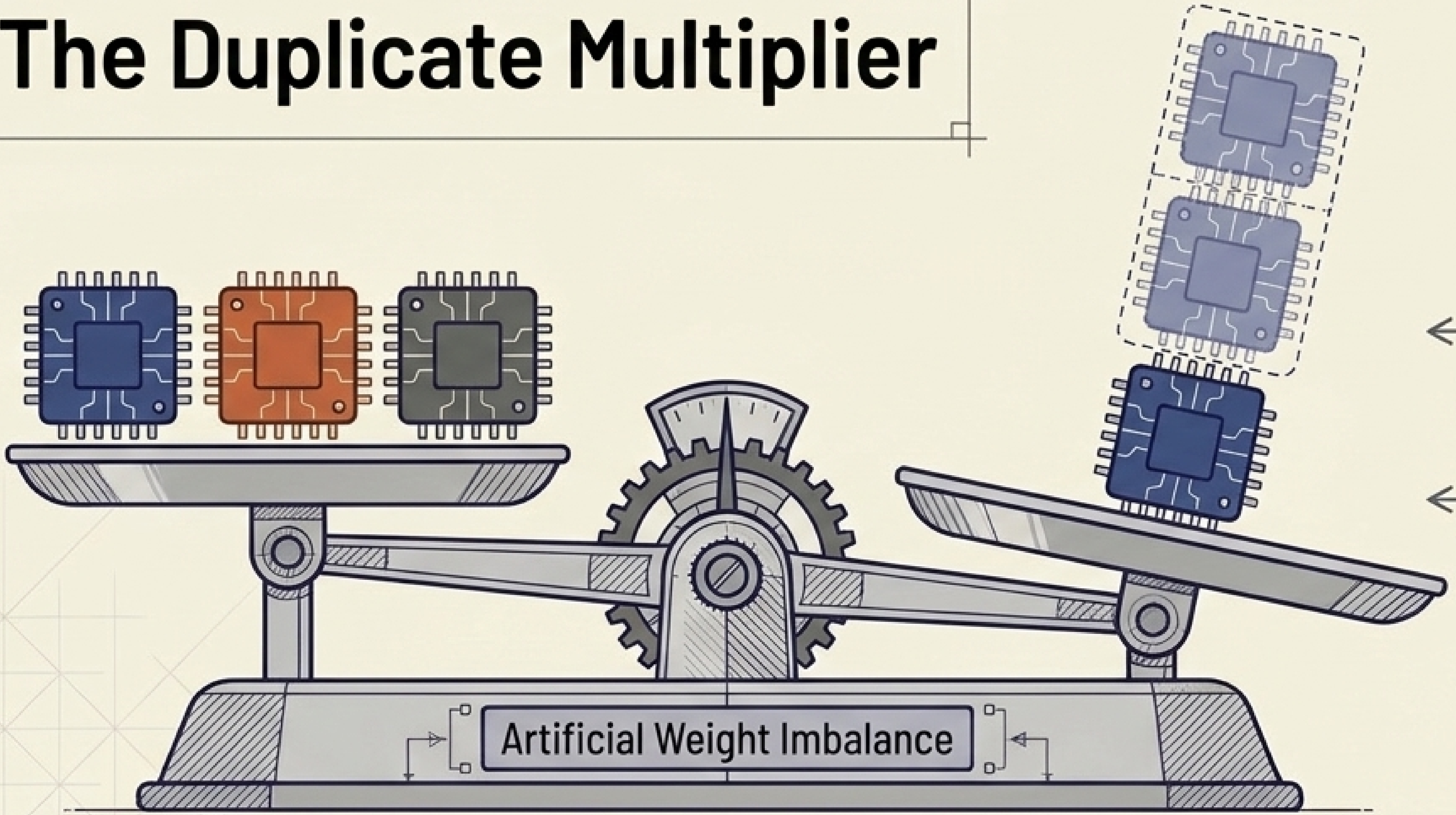
Integer Type

Integer Type

Triage: Handling Missing Data



The Duplicate Multiplier



Why do duplicates matter?

They unintentionally give certain observations more influence, skewing the model's baseline.

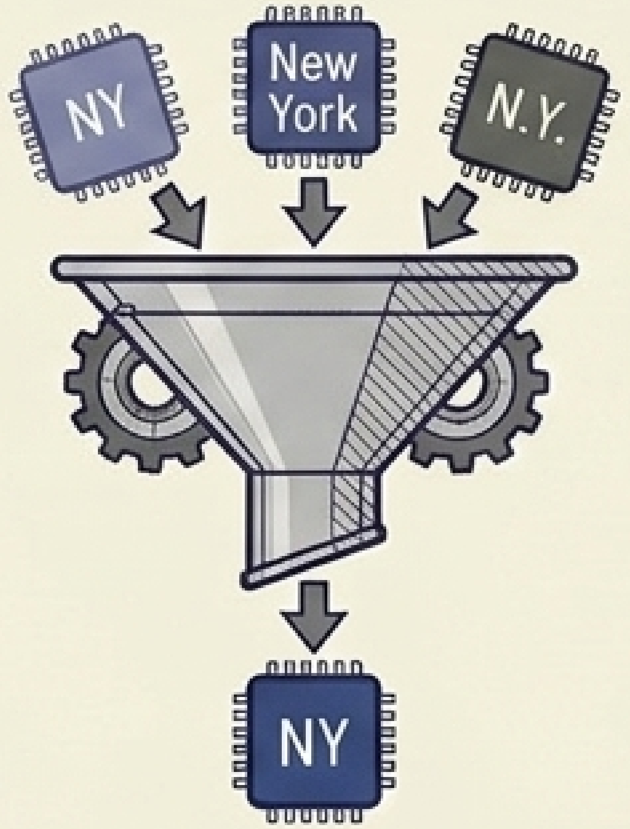
Find duplicates → Count occurrences → Remove (Drop) redundant rows

Validation: Numeric ≠ Valid

The Domain Error

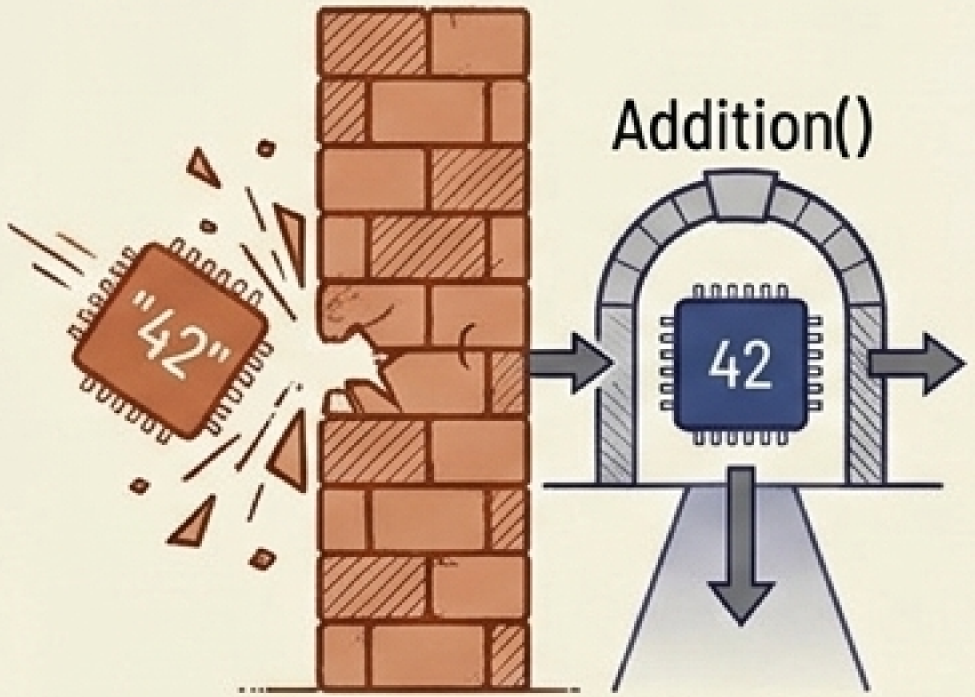
Technically a number.
Practically impossible. Requires domain knowledge to fix.

The Inconsistency



A model sees these as 3 different things.
Consistency matters.

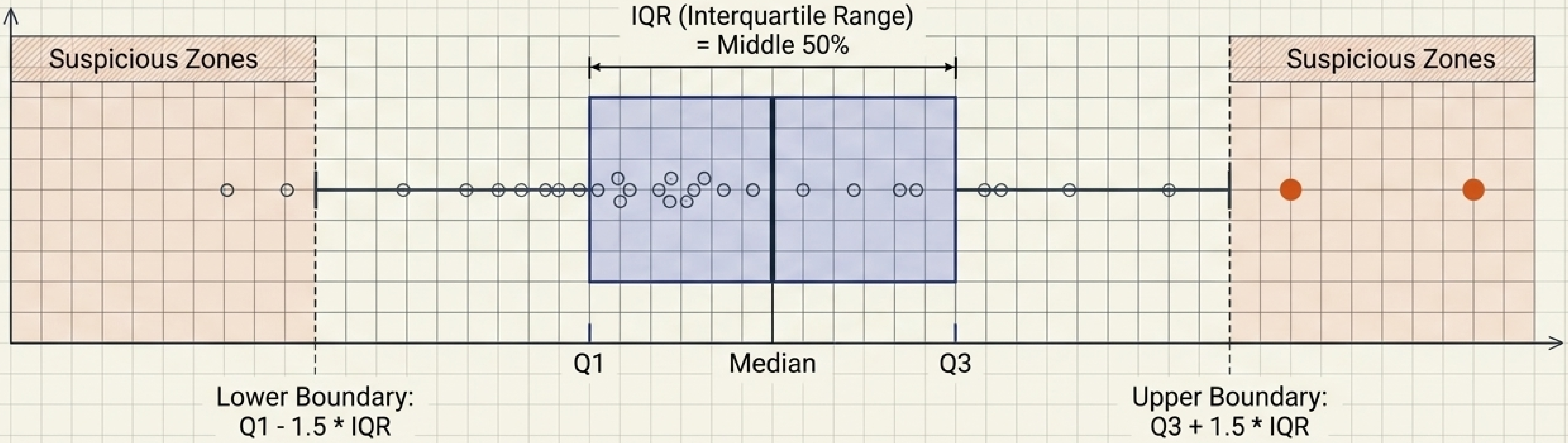
The Data Type Wall



Math operations require appropriate numerical data types.

Detecting Outliers (The IQR Boundary)

Rule: Potential outlier ≠ automatically bad data. (e.g., `Age = 900` is an error, but a sudden stock spike might be a genuine rare event. Never delete automatically).



Feature Encoding Translation Matrix

Machine Learning algorithms require numerical representations.

One-Hot Encoding

Mechanism

Converts categories into separate binary numerical features (0s and 1s).

When to Use

Unordered categories.

Visual Example

City
Paris
Tokyo
London

Transform →

is_Paris	is_Tokyo	is_London
1	0	0
0	1	0
0	0	1

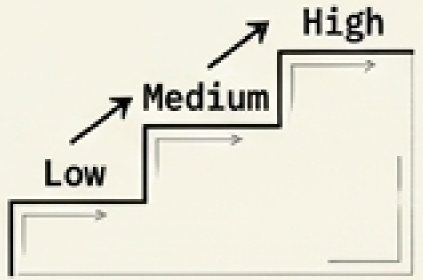
Label Encoding

Mechanism

Assigns a sequential integer to each category (1, 2, 3).

When to Use

Categories with a genuine order.



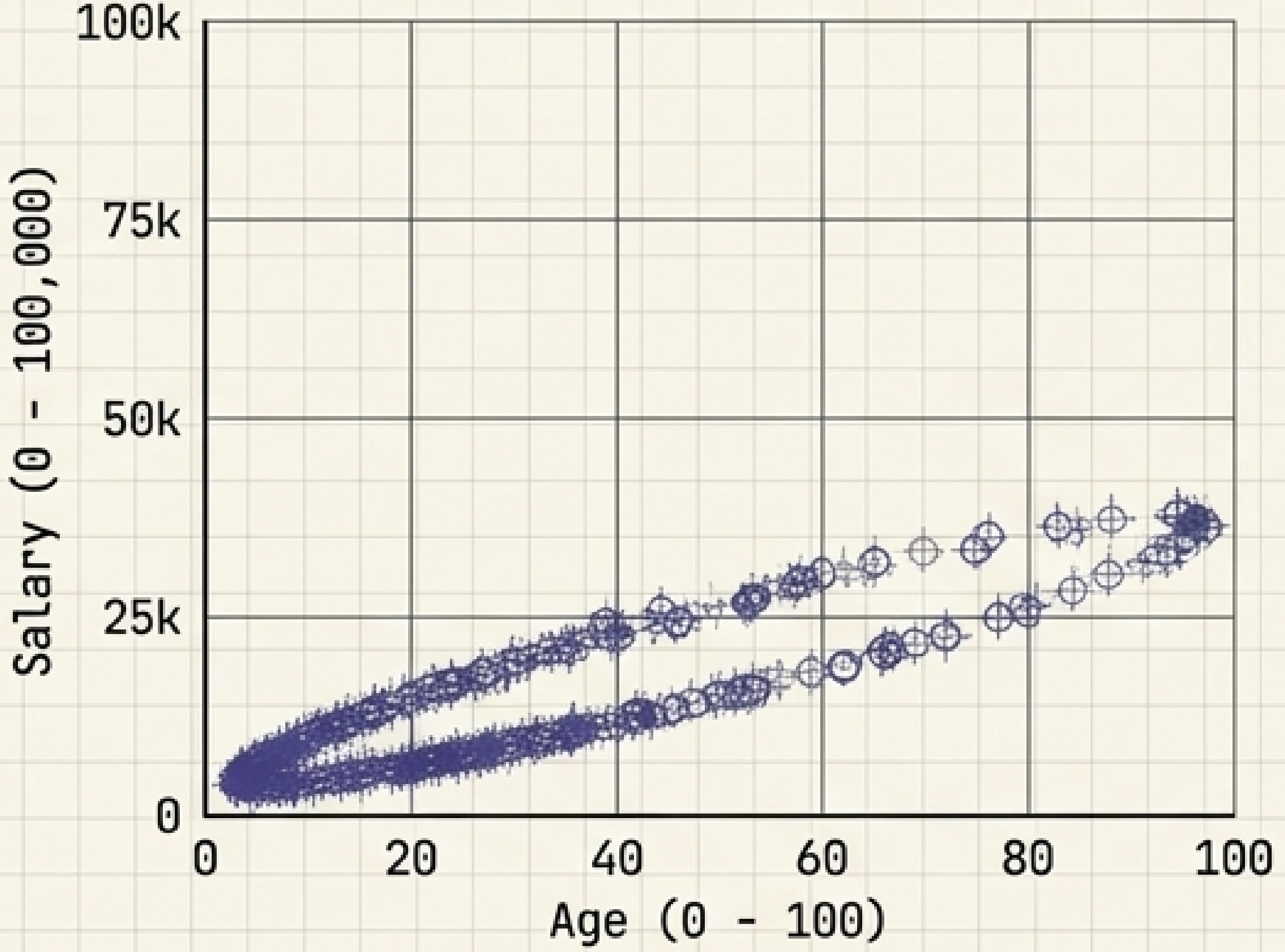
The Danger

Blindly assigning numbers introduces a false mathematical relationship. Green = 3, Blue = 2, Red = 1 does NOT mean Green > Blue > Red.

Feature Scaling: Fixing the Geometry

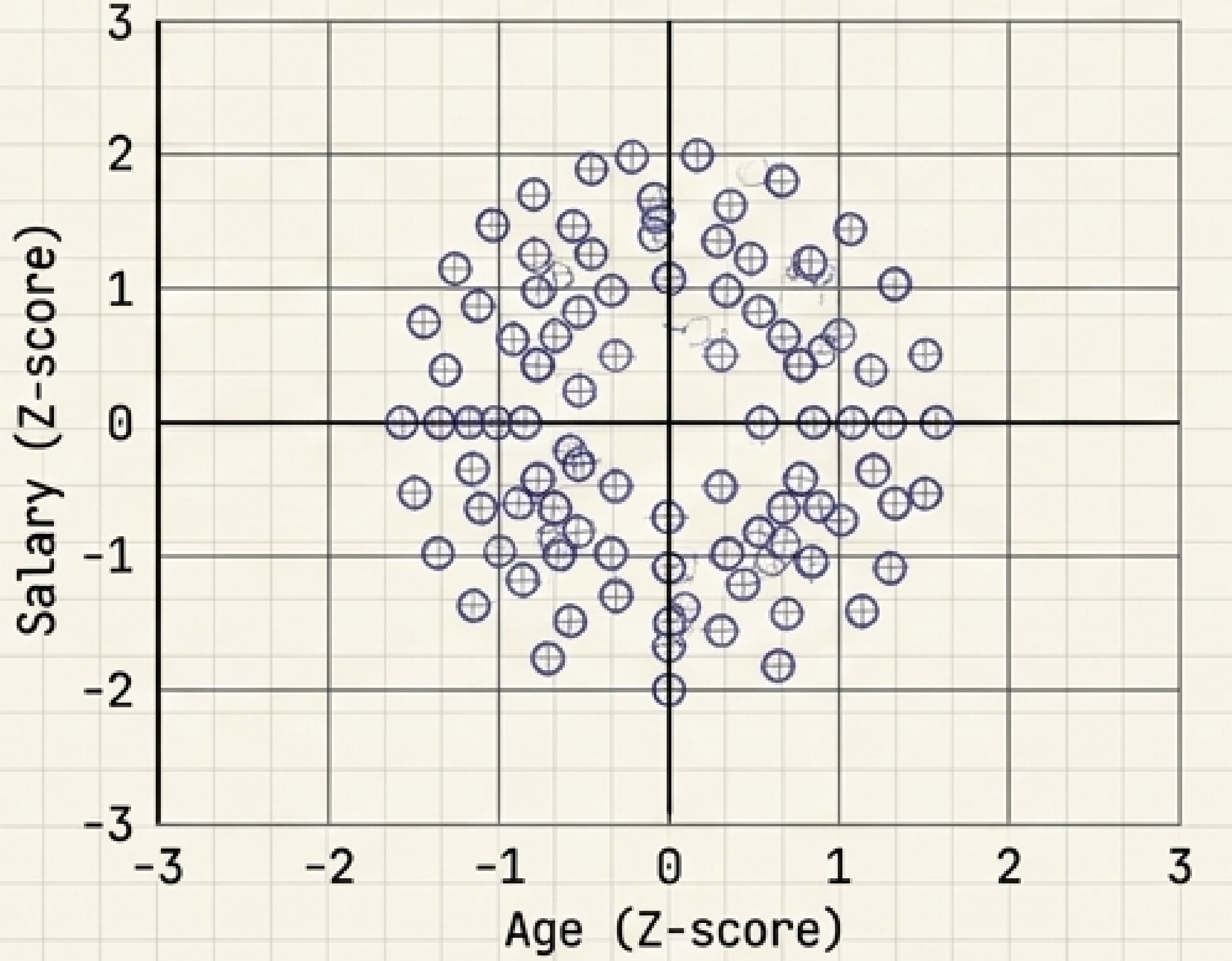
Algorithms are highly sensitive to feature scales.

Unscaled Data



Transformation

Standardized Data



Standardization transforms features to ~0 mean and unit variance.

td = original value

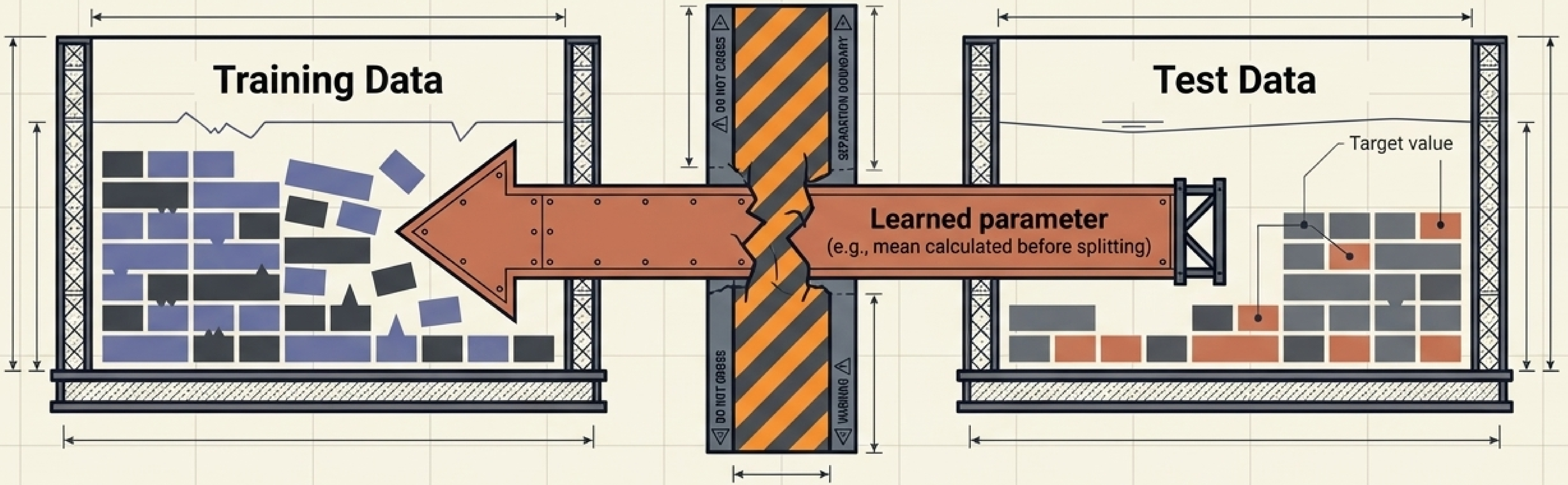
Formula: Z = (x - mean) / std

x = original value

mean = average of feature

The Hazard: Data Leakage

One of the most dangerous problems. Never let future or evaluation information leak into training.

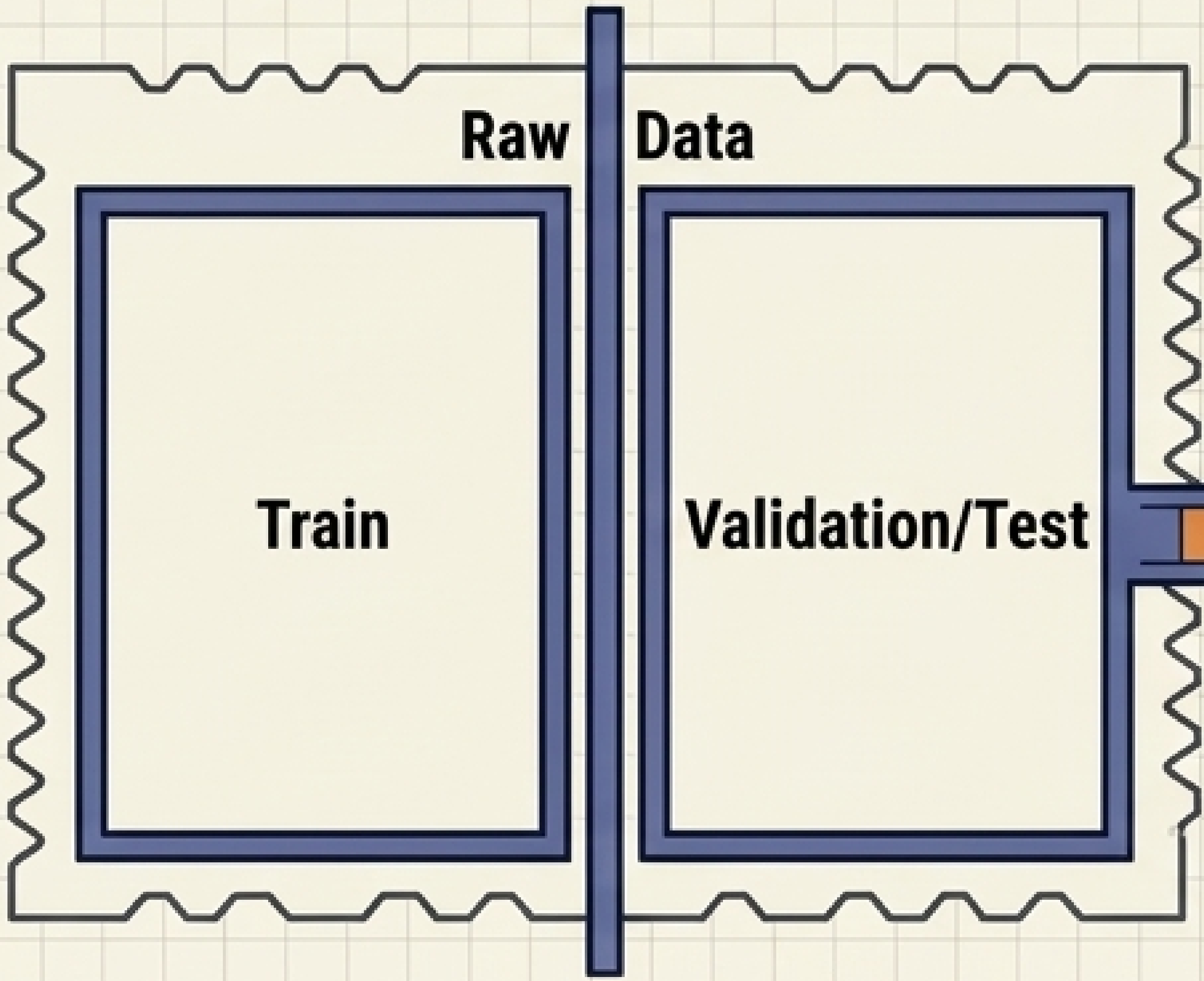


Result: The model has indirectly seen information from the test set. Expect overly optimistic performance and real-world failure.

The Correct Preprocessing Architecture

Step 1: The Split (First)

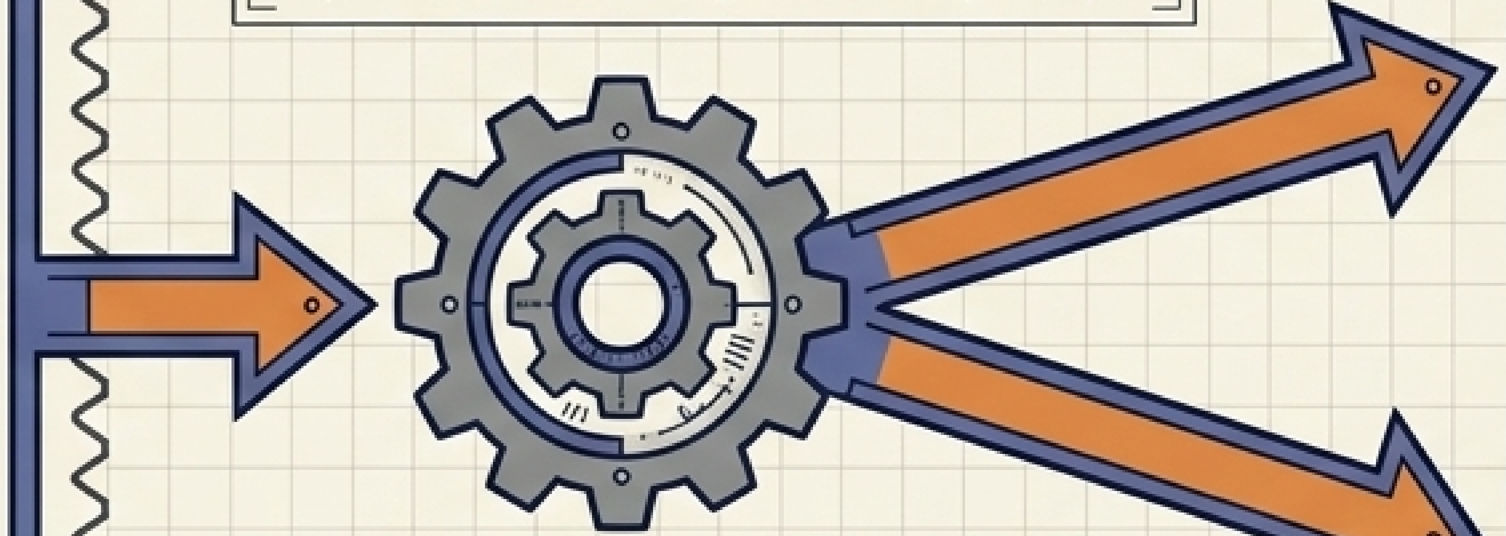
This happens *before* any other action.



Step 2: Learn on Train Only

Calculations for imputation (mean/median) or scaling parameters are extracted strictly from the Training block.

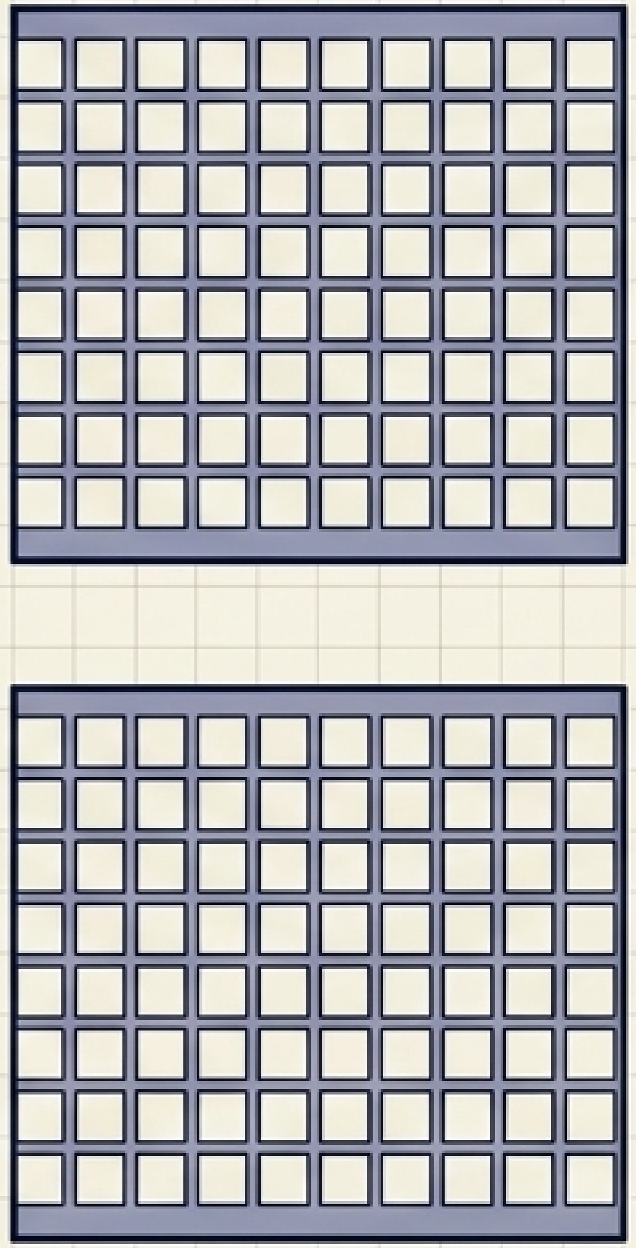
```
params = calculate_statistics(Train)
```



Transform using the exact same parameters.

```
Train_scaled = transform(Train, params)
Test_scaled = transform(Test, params)
```

Step 3: Apply to Both



Core Insight: Order of operations is just as important as the operations themselves.

The Mindset Shift

Data preprocessing is:
~~"Make the dataset look pretty."~~

Data preprocessing is:
"Make the data meaningful, consistent, and suitable for the learning algorithm—without leaking information."

Mini Practice: The Refinery Checklist

ITEM	DESCRIPTION	STATUS
TOD._..	--	B.E.
☑	1. Find missing_values.	
☑	2. Fill missing Age (Numerical logic).	
☑	3. Fill missing Salary (Numerical logic).	
☑	4. Find and drop duplicate_rows.	
☑	5. Standardize inconsistent city names (Categorical logic).	
☑	6. Inspect the final model-ready DataFrame.	

APPROVED FOR MODELING

DATE: ___/___

Use this checklist as your baseline for every new dataset.